

DATA 260 - Homework 1

Rental Housing Listings — Akanksha Shukla

Value	Result	Calculation
SID4	7838	Last four digits of student ID
PORT_BASE	8638	$8000 + (7838 \bmod 900)$
PREFIX	s7838	"s" + SID4
SEED	7838	SID4
VERIFY_SEED	267838	$260000 + \text{SID4}$
DOMAIN_ID	6	$7838 \bmod 8$ (Rental Housing Listings)

Hardware: MacBook Pro, Apple M4 Pro, 48 GB RAM

Local model used (Parts 2-4): qwen2:7b, served through Ollama

Tagged commit hash: hw1 (see repository)

Part 1: HTML, JavaScript, and Deployment

Domain and schema

Assigned domain: Rental Housing Listings (DOMAIN_ID 6). Before writing any HTML, DOMAIN_SCHEMA.md was created to define the entity's fields: propertyTitle, propertyLocation, submitterEmail, propertyDescription, propertyCategory, and termsAccepted, along with the four category values (Apartment, House, Condo, Townhouse) and the validation rules.

HTML

The page is titled HW1-Akanksha and uses an h1 tag reading "Rental Housing Listing" as the largest heading. The form includes: a required, autofocused text input for the property title (primary field), a required text input for the property location (secondary field), a required email input for the submitter's email, a required textarea for the property description, a dropdown with four categories, a checkbox labeled "I agree to the terms and conditions.", and a submit button labeled "Publish Rental Listing". The script tag is placed at the end of the body.

JavaScript

An arrow function (validateForm) checks that the description is longer than 25 characters and that the terms checkbox is checked, alerting the user if either rule fails. On successful submission the form data is converted to a JSON string with JSON.stringify and logged, then parsed back with JSON.parse. Object destructuring pulls out propertyTitle and submitterEmail and logs them individually. The spread operator copies the parsed object and adds a submissionDate field, and a closure (submissionCounter) keeps a private count that increments and is logged on every successful submission.

Screenshot: form submission (local file)

File

/Users/akanksha/Downloads/data260-7838/code/web_application/index.html

e.g., 123 Market St, San Jose, CA

Submitter email *

e.g., owner@example.com

Property description *

Describe the rooms, amenities, rent, and nearby attractions (more than 25 characters).

0 characters

Property category *

Select a category

☐ I agree to the terms and conditions.

Publish Rental Listing

Listing submitted successfully. Submission #2

Successful submissions

Spacious 2BR Apartment in Downtown San Jose — Apartment, 123 Market St, San Jose, CA

Delete

abc property — Apartment, Lkw

Delete

The form accepts input, validates it, and adds the new listing to the Successful submissions list after publishing.

Screenshot: console output for the same submission

```
Form data as JSON: {"propertyTitle":"abc property", "propertyLocation":"Lkw", "submitterEmail":"shlvishu93@gmail.com", "propertyDescription":"near downtown, with 1 balcony, renovated and fully furnished.", "propertyCategory":"Apartment", "termsAccepted":"agreed"} script.js:144

Parsed form data: {propertyTitle: 'abc property', propertyLocation: 'Lkw', submitterEmail: 'shlvishu93@gmail.com', propertyDescription: 'near downtown, with 1 balcony, renovated and fully furnished.', propertyCategory: 'Apartment', ...} script.js:148

Primary field (property title): abc property script.js:153

Submitter email: shlvishu93@gmail.com script.js:154

Updated parsed object: {propertyTitle: 'abc property', propertyLocation: 'Lkw', submitterEmail: 'shlvishu93@gmail.com', propertyDescription: 'near downtown, with 1 balcony, renovated and fully furnished.', propertyCategory: 'Apartment', ...} script.js:165

Successful submission count: 2 script.js:166

Listing "abc property" saved successfully. script.js:173

> cmd i to turn on code suggestions. Don't show again NEW
```

Console shows the JSON string, the parsed object, the destructured title and email, the updated object after the spread operator, and the submission count from the closure.

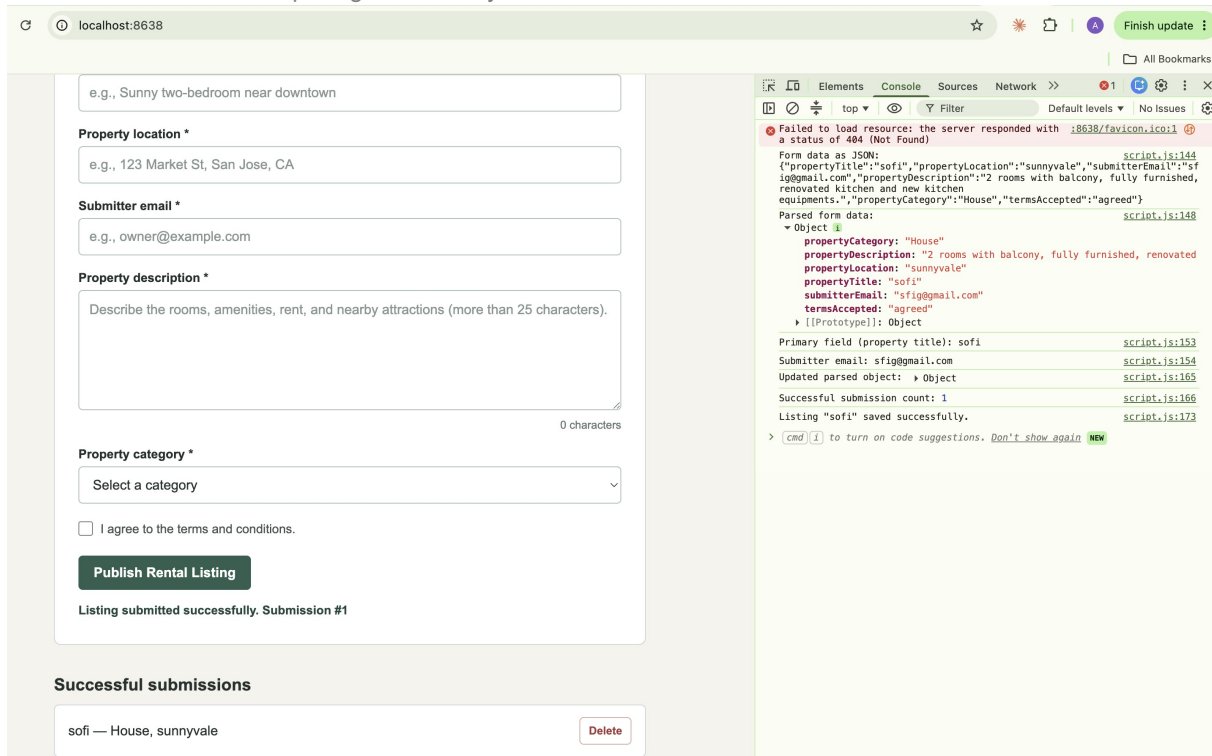
Docker deployment

The application is built into a Docker image using an Nginx Alpine base image (hw1-rental-listings:1.0) and run as a container named s7838-rental-housing, published on port 8638 (PORT_BASE), matching the calculated configuration value.

```
akanksha@Akankshas-MacBook-Pro data260-7838 % cd /Users/akanksha/Downloads/data260-7838
docker build -f code/Dockerfile -t hw1-rental-listings:1.0 .
docker run --detach --name s7838-rental-housing --publish 8638:80 hw1-rental-listings:1.0
[+] Building 0.5s (10/10) FINISHED
[=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 605B 0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 0.4s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 161B 0.0s
=> [1/5] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d 0.0s
=> => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 218B 0.0s
=> CACHED [2/5] RUN rm -rf /usr/share/nginx/html/* 0.0s
=> CACHED [3/5] COPY code/web_application/index.html /usr/share/nginx/html/index.html 0.0s
=> CACHED [4/5] COPY code/web_application/styles.css /usr/share/nginx/html/styles.css 0.0s
=> CACHED [5/5] COPY code/web_application/script.js /usr/share/nginx/html/script.js 0.0s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => exporting manifest sha256:a56c62943276c15ec7065a45ece5a32790b5f32adc0d35acd668691e32936ad 0.0s
=> => exporting config sha256:2571ba4ace3120679df529ee998dd3c7b4646749080a3ffd62d2eb451ba01779 0.0s
=> => exporting attestation manifest sha256:4bb93d4218a2d220fa567a4a5db255afc5f8184c4d47a693f8e2b9151a2f5ad5 0.0s
=> => exporting manifest list sha256:47a1bcb043ef8394c5fd30f46495d012afe8c2d225dd2b9ce420153ca86e3e9 0.0s
=> => naming to docker.io/library/hw1-rental-listings:1.0 0.0s
=> => unpacking to docker.io/library/hw1-rental-listings:1.0 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/n0xyv0y9mzceih7wxl5qodmr
4498907a1ed553ea607c5a819824f34538888a9ea08d644cd23a005c9ccf4784
akanksha@Akankshas-MacBook-Pro data260-7838 %
```

docker build and docker run completing successfully.



The app running at localhost:8638 inside the container, with the same console behavior verified again.

AWS ECS deployment

The same Docker image was deployed as a single-task AWS ECS service. The screenshot below shows the form loaded and a listing submitted successfully from the task's public IP address (32.192.189.237), confirming the container is reachable outside localhost.

Not Secure 32.192.189.237

e.g., Sunny two-bedroom near downtown

Property location *

e.g., 123 Market St, San Jose, CA

Submitter email *

e.g., owner@example.com

Property description *

Describe the rooms, amenities, rent, and nearby attractions (more than 25 characters).

0 characters

Property category *

Select a category

☐ I agree to the terms and conditions.

Publish Rental Listing

Listing submitted successfully. Submission #1

Successful submissions

abc property — Condo, sunnyvale [Delete](#)

App running at the ECS task's public IP.

Part 2: Agentic AI (Planner, Reviewer, Finalizer)

Command used: `python agents_demo.py` (run from the `code/` directory, with Ollama serving `qwen2:7b` on `localhost:11434`).

`agents_demo.py` sends the input title and content to a Planner agent, which returns 3 tags and a summary as strict JSON (format="json" on the Ollama call). A Reviewer agent then looks at the same title/content plus the Planner's output and either keeps it or improves it, marking `changed` true or false. A Finalizer step takes whichever output is better (Reviewer's if changed, Planner's otherwise), trims the tag list down to exactly 3, and checks the summary is at most 25 words. Neither prompt references the rental domain by name or by a fixed keyword list; both work only from the title and content passed in, so the same code would work for a different domain's listing.

Input used

Title: "Spacious 2BR Apartment in Downtown San Jose". Content: renovated 2-bedroom apartment, hardwood floors, in-unit washer/dryer, city view balcony, fitness center, rooftop garden, 24/7 security, pet-friendly, move-in special.

Screenshot: Planner output

```
INPUT TITLE: Spacious 2BR Apartment in Downtown San Jose
INPUT CONTENT: This beautifully renovated 2-bedroom, 1-bathroom apartment is located in the heart of downtown San Jose.
Features include hardwood floors, high ceilings, in-unit washer/dryer, and a large balcony with city views.
Building amenities include a fitness center, rooftop garden, and 24/7 security. Near public transit, restaurants,
and shopping. Pet-friendly with a $500 deposit. Move-in special: first month 50% off. Lease term flexible.

=====
PLANNER OUTPUT:
=====
{
  "tags": [
    "2BR Apartment",
    "Downtown San Jose",
    "Rental Offer"
  ],
  "summary": "Spacious, renovated 2-bedroom apartment in downtown San Jose with city views and building amenities, offer
ing a move-in special of 50% off the first month's rent.",
  "reasoning": "The title and content highlight the apartment's location, bedroom count, and special offer, which are th
e most relevant keywords for this content."
}
```

Screenshot: Reviewer output

```
REVIEWER OUTPUT:
=====
{
  "tags": [
    "2BR Apartment",
    "Downtown San Jose",
    "Rental Offer",
    "City Views",
    "Building Amenities",
    "Move-in Special"
  ],
  "summary": "Spacious, renovated 2-bedroom apartment with city views in downtown San Jose, offering building amenities and a move-in special of 50% off the first month's rent.",
  "changed": true,
  "explanation": "Added 'City Views', 'Building Amenities', and 'Move-in Special' to provide more specific details and benefits."
}
```

Screenshot: Finalizer step

```
FINALIZER STEP:
=====
{
  "tags": [
    "2BR Apartment",
    "Downtown San Jose",
    "Rental Offer"
  ],
  "summary": "Spacious, renovated 2-bedroom apartment with city views in downtown San Jose, offering building amenities and a move-in special of 50% off the first month's rent.",
  "word_count": 25,
  "source": "Reviewer"
}
```

Screenshot: final published JSON

```
=====
FINAL PUBLISH OUTPUT (VALID JSON):
=====
{
  "tags": [
    "2BR Apartment",
    "Downtown San Jose",
    "Rental Offer"
  ],
  "summary": "Spacious, renovated 2-bedroom apartment with city views in downtown San Jose, offering building amenities and a move-in special of 50% off the first month's rent.",
  "word_count": 25,
  "source": "Reviewer"
}
(env) akanksha@Akankshas-MacBook-Pro code %
```

Q1. The 3 final tags obtained

2BR Apartment, Downtown San Jose, Rental Offer

Q2. The final summary (at most 25 words)

"Spacious, renovated 2-bedroom apartment with city views in downtown San Jose, offering building amenities and a move-in special of 50% off the first month's rent."

Q3. Did the Reviewer agent change anything?

Yes. The Reviewer kept the Planner's original 3 tags but added City Views, Building Amenities, and Move-in Special, and marked changed: true with the explanation "Added 'City Views', 'Building Amenities', and

'Move-in Special' to provide more specific details and benefits." The Finalizer then cut the Reviewer's 6 tags back down to the required 3.

Part 3: Measuring Non-Determinism

The same `agents_demo.py` pipeline was run 40 times on one fixed input (saved unchanged as `reports/hw01/cases/nondeterminism_input.json`): 20 runs at temperature 0.7 and 20 runs at temperature 0.0. `agents_demo.py` was extended to accept an input file path and a temperature as command-line arguments, so the exact same pipeline code used in Part 2 produced every run.

Command used

`python run_nondeterminism_tests.py`, which calls `python agents_demo.py reports/hw01/cases/nondeterminism_input.json <temperature>` forty times and saves every result to `reports/hw01/raw/nondeterminism_results.json`. Then `python analyze_nondeterminism.py` computes the statistics below and writes `reports/hw01/METRICS.md`.

Screenshot: start of the 40-run test

```
Last login: Sun Aug 30 17:08:39 on ttys007
akanksha@Akankshas-MacBook-Pro code % cd /Users/akanksha/Downloads/data260-7838/code
source venv/bin/activate
date | tee -a ../reports/hw01/RUN_LOG.txt
python run_nondeterminism_tests.py | tee -a ../reports/hw01/RUN_LOG.txt
Sun Aug 30 23:10:48 PDT 2026
```

Screenshot: all 40 runs completing


```
Last login: Sun Aug 30 23:10:45 on ttys008
akanksha@Akankshas-MacBook-Pro code % cd /Users/akanksha/Downloads/data260-7838/code
source venv/bin/activate
date | tee -a ../reports/hw01/RUN_LOG.txt
python run_nondeterminism_tests.py | tee -a ../reports/hw01/RUN_LOG.txt
Sun Aug 30 23:23:58 PDT 2026
Part 3: Non-Determinism Testing (40 runs total)

Input file: ../reports/hw01/cases/nondeterminism_input.json

Running 20 tests at temperature 0.7 (creative)...
Running test 1/40 at temperature 0.7... 5591ms
Running test 2/40 at temperature 0.7... 4105ms
Running test 3/40 at temperature 0.7... 4014ms
Running test 4/40 at temperature 0.7... 3664ms
Running test 5/40 at temperature 0.7... 5019ms
Running test 6/40 at temperature 0.7... 4267ms
Running test 7/40 at temperature 0.7... 3940ms
Running test 8/40 at temperature 0.7... 4143ms
Running test 9/40 at temperature 0.7... 4659ms
Running test 10/40 at temperature 0.7... 3914ms
Running test 11/40 at temperature 0.7... 4776ms
Running test 12/40 at temperature 0.7... 4066ms
Running test 13/40 at temperature 0.7... 4311ms
Running test 14/40 at temperature 0.7... 4501ms
Running test 15/40 at temperature 0.7... 4601ms
Running test 16/40 at temperature 0.7... 3666ms
Running test 17/40 at temperature 0.7... 4753ms
Running test 18/40 at temperature 0.7... 5232ms
Running test 19/40 at temperature 0.7... 4383ms
Running test 20/40 at temperature 0.7... 4240ms

Running 20 tests at temperature 0.0 (deterministic)...
Running test 21/40 at temperature 0.0... 4149ms
Running test 22/40 at temperature 0.0... 3950ms
Running test 23/40 at temperature 0.0... 3963ms
Running test 24/40 at temperature 0.0... 3951ms
Running test 25/40 at temperature 0.0... 3961ms
Running test 26/40 at temperature 0.0... 3953ms
Running test 27/40 at temperature 0.0... 3946ms
Running test 28/40 at temperature 0.0... 3961ms
Running test 29/40 at temperature 0.0... 4032ms
Running test 30/40 at temperature 0.0... 3894ms
Running test 31/40 at temperature 0.0... 3843ms
Running test 32/40 at temperature 0.0... 3861ms
Running test 33/40 at temperature 0.0... 3849ms
Running test 34/40 at temperature 0.0... 3843ms
Running test 35/40 at temperature 0.0... 3859ms
Running test 36/40 at temperature 0.0... 3844ms
Running test 37/40 at temperature 0.0... 3864ms
Running test 38/40 at temperature 0.0... 3850ms
Running test 39/40 at temperature 0.0... 3856ms
Running test 40/40 at temperature 0.0... 3860ms

Results saved to: ../reports/hw01/raw/nondeterminism_results.json
Total runs completed: 40
```

Screenshot: analysis step and generated METRICS.md

```
[(venv) akanksha@Akankshas-MacBook-Pro code % python analyze_nondeterminism.py
Loaded 40 results
Results saved to: ../reports/hw01/METRICS.md

Analysis complete.
Metrics file: reports/hw01/METRICS.md
Raw data: reports/hw01/raw/nondeterminism_results.json

[(venv) akanksha@Akankshas-MacBook-Pro code % cat ../reports/hw01/METRICS.md
# Part 3: Measuring Non-Determinism - Results

**Domain**: Rental Housing Listings (DOMAIN_ID 6)
**Input**: Spacious 2BR Apartment in Downtown San Jose
**Model**: qwen2:7b
**Total Runs**: 40 (20 at temp 0.7, 20 at temp 0.0)

---

## Metrics Table

| Metric | Temp 0.0 | Temp 0.7 |
|-----|-----|-----|
| Distinct tag sets | 1 | 14 |
| Tags in all 20 runs | 3 | 0 |
| Tags in exactly 1 run | 0 | 6 |

---

## Latency Analysis

| Metric | Temp 0.0 | Temp 0.7 |
|-----|-----|-----|
| p50 (ms) | 3879.1 | 4289.1 |
| p95 (ms) | 4143.3 | 5573.2 |
| p99 (ms) | 4241.4 | 5874.8 |

---

## Detailed Analysis

### Temperature 0.0 (Deterministic)

**Distinct tag sets**: 1

**Tags appearing in ALL 20 runs**:
- Downtown San Jose
- Pet-friendly
- Spacious 2BR

**Tags appearing in EXACTLY 1 run**:
- None

---

### Temperature 0.7 (Creative/Non-Deterministic)

**Distinct tag sets**: 14
```

```
**Tags appearing in ALL 20 runs**:
- None
```

```
**Tags appearing in EXACTLY 1 run**:
- CityViews
- FlexibleLease
- Rooftop Garden
- Spacious 2-bedroom apartment
- Spacious Apartment
- SpaciousApartment
```

Results

Metric	Temp 0.0	Temp 0.7
Distinct tag sets	1	14
Tags in all 20 runs	3	0
Tags in exactly 1 run	0	6
Metric	Temp 0.0	Temp 0.7
p50 (ms)	3879.1	4289.1
p95 (ms)	4143.3	5573.2
p99 (ms)	4241.4	5874.8

At temp 0.0, all 20 runs produced the identical tag set (Downtown San Jose, Pet-friendly, Spacious 2BR) fully reproducible. At temp 0.7, only 14 of 20 runs shared a tag set, and 6 tags (CityViews, FlexibleLease, Rooftop Garden, and three variants of "Spacious apartment") each appeared in only one run.

What two users sending identical input might see

At temperature 0.7, two real runs on the identical input produced: ['SpaciousApartment', 'CityViews', 'FlexibleLease'] versus ['Spacious 2BR Apartment', 'Downtown San Jose', 'Pet-friendly']. Both describe the same listing, but with different word choices and different aspects emphasized, because temperature above 0 makes the model sample among likely next words instead of always choosing the single most probable one. At temperature 0.0, both users get the identical tag set every time, since the model always picks the most probable token and there is no randomness left to vary the output.

Example where variation is acceptable

Browsing/search tagging, like this exact use case. If one user's results show "Pet-friendly" and another's show "Animal-Friendly," it does not change their decision to view the listing - the meaning is preserved even if the wording differs.

Example where variation is NOT acceptable

Calculating a number, such as monthly rent after a stated percentage increase. The answer must be identical every time it is computed from the same inputs, so temperature 0.0 is required; any variation would be a bug, not a feature.

Part 4: Model Client and Token Accounting

src/model_client.py defines ModelClient with a stable complete(messages, tools=None) interface; every model call in the demo goes through this method. hw1_client.py is an interactive command-line demo that imports the adapter: the user types messages, and the adapter tracks input/output/total tokens per turn plus cumulative totals across the session. Typing /stats prints turn count, cumulative token counts, and the length of the serialized (JSON) conversation history, without calling the model or altering that history.

Code review against AGENT.md

AGENT.md asks for a strict, bullet-only review of `src/model_client.py` and `hw1_client.py` covering correctness, token tracking, design quality, and error handling. Running that review surfaced a real bug: `complete()` was appending the incoming messages list onto `self.conversation_history` on every call, but the caller also reconstructs the full history each turn and passes it back in - so history (and therefore input token counts) was roughly doubling every turn instead of growing linearly. It was fixed so `complete()` replaces history with exactly what was sent plus the new reply. A second issue - `hw1_client.py`'s ``from src.model_client import ModelClient`` failed with `ModuleNotFoundError` when run normally, since `src/` was not on the import path - fixed by inserting the repo root into `sys.path`. A third issue `conversation_history_length` summed raw message text instead of a serialized length - fixed to measure `len(json.dumps(...))` of the history instead.

Command used

`python hw1_client.py` (run from the `code/` directory, with Ollama serving `qwen2:7b`). The conversation covered the Rental Housing Listings domain: what to include in a listing description, amenities that stand out, fixed-term vs. month-to-month leases, factors affecting rent, and red flags to watch for in a listing. `/stats` was invoked after turn 3 and after turn 5.

Screenshot: /stats after turn 3

```
You: /stats

/stats (after turn 3):
  Turn Count: 3
  Cumulative Input Tokens: 1282
  Cumulative Output Tokens: 987
  Cumulative Total Tokens: 2269
  Serialized Conversation History Length: 4609 chars
```

Screenshot: /stats after turn 5

```
You: /stats

/stats (after turn 5):
  Turn Count: 5
  Cumulative Input Tokens: 4028
  Cumulative Output Tokens: 2187
  Cumulative Total Tokens: 6215
  Serialized Conversation History Length: 9726 chars
```

Screenshot: final cumulative summary on exit

```
FINAL SUMMARY

=====

Conversation complete.
Total Turns: 5
Total Input Tokens: 4028
Total Output Tokens: 2187
Total Tokens Used: 6215
Serialized Conversation History Length: 9726 chars
(venv) akanksha@Akankshas-MacBook-Pro code %
```

Results

Checkpoint	Turn Count	Cum. Input	Cum. Output	Cum. Total	History (chars)
After turn 3	3	1282	987	2269	4609
After turn 5 / exit	5	4028	2187	6215	9726

Per-turn input tokens grew across the conversation (56, 462, 764, 1094, 1652 for turns 1-5), confirming linear growth from resending history each turn rather than the exponential growth that the pre-fix duplication bug would have produced.

Understanding Tokens and Context

Why is prior conversation context resent with every turn?

The model itself is stateless. It has no memory between separate API/invoke calls. The only way it can respond consistently with earlier turns is if the caller resends the full prior conversation (system prompt, all previous user messages, and all previous assistant replies) alongside the new message every single time.

How is a system prompt different from a user message?

A system prompt sets the assistant's behavior and rules ("be concise, answer directly") and is written by the application, not the end user. A user message is the actual question or request from the person using the app. Both count as input tokens and both get resent every turn, but a system prompt is an instruction to the model while a user message is data for the model to respond to.

Why do input tokens grow over a conversation?

Because each turn's input is the system prompt plus every prior user and assistant message plus the new user message. Nothing already sent is ever removed, so the input keeps growing turn over turn. This run measured it directly: input tokens per turn were 56, 462, 764, 1094, then 1652 for turns 1 through 5 climbing every turn because turn N's input contains all of turns 1 through N-1.

What eventually limits that growth?

The model's context window is the hard limit - qwen2:7b can only accept a fixed maximum number of tokens per call, and once the resent history plus the new message exceeds that, older turns would have to be dropped or summarized. Before hitting that hard limit, cost and latency are practical limits: more tokens sent per turn means slower responses and, on paid APIs, higher cost per turn, which is why production systems often truncate, summarize, or cap history length rather than resending an ever-growing transcript indefinitely.